LESSON 5

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

# Rainbow Colors

Dim LEDs smoothly and mix any colour you like.

Name: _____ Date: _____

### TODAY YOU WILL LEARN

- How **PWM** fakes "half on" with very fast blinking
- How to dim an LED with `analogWrite`
- How to fade with **for loops** that count up and down
- How an **RGB LED** mixes red, green and blue light

### SESSION PLAN

| | |
|---|---|
| 0:00 | Warm-up: how do screens make colours? |
| 0:10 | PWM: blinking too fast to see |
| 0:20 | Sketch 1: **L05_Fade** |
| 0:40 | Build the RGB LED circuit |
| 0:55 | Short break |
| 1:00 | Sketch 2: **L05_ColorMixer** |
| 1:20 | Sketch 3: **L05_KnobColor** |
| 1:35 | Challenges |
| 1:55 | Quiz and tidy up |

### PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Your yellow LED on pin 26 (from Lesson 2)
- ☐ 1 RGB LED (4 legs)
- ☐ 3 resistors, 220 Ω (red-red-brown)
- ☐ Your potentiometer on pin 34 (from Lesson 4)
- ☐ 4 jumper wires

## 1. Blinking too fast to see

`digitalWrite` can only switch an LED fully **on** or fully **off**. So how do we make it **half** bright?

The trick is to blink it **very fast**: thousands of times a second. Your eyes can't follow that, so they see the *average*. On for half the time looks half bright. On for a tenth of the time looks very dim. This trick is called **PWM** (Pulse Width Modulation).

| On for… | Looks… | `analogWrite` value |
|---|---|---|
| 0% of the time | off | 0 |
| 25% of the time | dim | 64 |
| 50% of the time | half bright | 128 |
| 100% of the time | fully on | 255 |

> **Big idea: 0 to 255**
>
> `analogWrite(pin, value)` takes a value from **0** (off) to **255** (fully on). Why 255? It's the biggest number that fits in 8 bits (one byte) of computer memory.

## 2. Sketch 1: Fade in, fade out

Keep the yellow LED on pin 26 from Lesson 2. **File → New Sketch**, save it as **L05_Fade**, and upload.

Type this in → save it as **L05_Fade** — 19 lines

```
// Lesson 5: Fade an LED in and out
// Yellow LED on pin 26 (from Lesson 2).

const int LED = 26;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  for (int brightness = 0; brightness <= 255; brightness++) {
    analogWrite(LED, brightness);
    delay(5);
  }
  for (int brightness = 255; brightness >= 0; brightness--) {
    analogWrite(LED, brightness);
    delay(5);
  }
}
```

### How it works

| Code | What it means |
|---|---|
| `analogWrite(LED, brightness);` | Set the LED's brightness, from 0 to 255 |
| `for (int brightness = 0; brightness <= 255; brightness++)` | Count **up** from 0 to 255 |
| `for (int brightness = 255; brightness >= 0; brightness--)` | Count **down** from 255 to 0 |
| `brightness--` | Take 1 away from `brightness` |
| `delay(5);` | Wait a tiny bit at each step, so the fade is slow enough to see |

**Try it**

1. How long does one fade up take? *Hint: 256 steps × 5 ms.*

---

2. Change `delay(5)` to `delay(1)`, then to `delay(20)`. Which looks best?

## 3. Build the RGB LED circuit

An **RGB LED** is really three tiny LEDs in one: **R**ed, **G**reen and **B**lue. It has four legs. The **longest** leg is the shared **–** leg (called the common cathode). It goes to GND.

> **Which leg is which?**
>
> Hold the LED with the longest leg second from the left. From left to right the legs are usually: **Red**, **GND (longest)**, **Green**, **Blue**. If your colours come out mixed up, just swap the wires on pins 25, 26 and 27.

**Take out your three traffic light LEDs** (but leave their resistors if you like) and add the RGB LED:

| ✓ | From | To |
|---|---|---|
| ☐ | RGB LED | each leg in its own row |
| ☐ | **longest** leg | **–** rail (GND) |
| ☐ | ESP32 pin **25** | 220 Ω resistor → **red** leg |
| ☐ | ESP32 pin **26** | 220 Ω resistor → **green** leg |
| ☐ | ESP32 pin **27** | 220 Ω resistor → **blue** leg |

Keep the potentiometer on pin 34 from Lesson 4. You'll need it in Sketch 3.

## 4. Sketch 2: Colour mixer

Screens make every colour by mixing red, green and blue light. Now you can do that too! **File → New Sketch**, save it as **L05_ColorMixer**.

Type this in → save it as **L05_ColorMixer** — 33 lines

```
// Lesson 5: Mix colours with an RGB LED
// RGB LED: red 25, green 26, blue 27 (220 ohm each), long leg GND.

const int RED = 25;
const int GREEN = 26;
const int BLUE = 27;

void setColor(int r, int g, int b) {
  analogWrite(RED, r);
  analogWrite(GREEN, g);
  analogWrite(BLUE, b);
}

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);
}
```

```
void loop() {
  setColor(255, 0, 0);      // red
  delay(1000);
  setColor(0, 255, 0);      // green
  delay(1000);
  setColor(0, 0, 255);      // blue
  delay(1000);
  setColor(255, 255, 0);    // yellow
  delay(1000);
  setColor(128, 0, 255);    // purple
  delay(1000);
  setColor(255, 255, 255);  // white
  delay(1000);
}
```

| Code | What it means |
|---|---|
| `void setColor(int r, int g, int b)` | Our own function. It sets all three colours in one go |
| `setColor(255, 255, 0);` | Full red + full green + no blue = **yellow** |
| `setColor(128, 0, 255);` | Half red + full blue = **purple** |

**Hard to see the colours?**

Put a small piece of white paper or a ping-pong ball over the LED. It blends the three colours together.

**Colour lab**

Try these mixes and write down what colour you see: `(0, 255, 255)` `(255, 0, 255)` `(255, 80, 0)` `(20, 20, 20)`

## 5. Sketch 3: Knob colour picker

Now let the knob choose the colour. As you turn it, the colour moves around the **colour wheel**: red → green → blue → back to red.

**File → Save As…** and name it **L05_KnobColor**. Change your code to the sketch below.

The knob gives a number 0–764, split into three parts of 255:

| `pos` | Colour change |
|---|---|
| 0 – 254 | red fades out while green fades in |
| 255 – 509 | green fades out while blue fades in |
| 510 – 764 | blue fades out while red fades in |

`else if` lets you test several things in a row: the first test that is true wins, and the rest are skipped.

Type this in → save it as **L05_KnobColor** 34 lines

```
// Lesson 5: Turn the knob to pick a colour
// RGB LED on 25/26/27, potentiometer middle leg on 34.

const int RED = 25;
const int GREEN = 26;
const int BLUE = 27;
const int KNOB = 34;

void setColor(int r, int g, int b) {
  analogWrite(RED, r);
  analogWrite(GREEN, g);
  analogWrite(BLUE, b);
}

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);
}

void loop() {
  int pos = map(analogRead(KNOB), 0, 4095, 0, 764);

  if (pos < 255) {
    setColor(255 - pos, pos, 0);          // red to green
  } else if (pos < 510) {
    pos = pos - 255;
    setColor(0, 255 - pos, pos);          // green to blue
  } else {
    pos = pos - 510;
    setColor(pos, 0, 255 - pos);          // blue to red
  }
  delay(20);
}
```

## 6. Challenges

### Challenge 1: Traffic light, RGB style

Use `setColor` to make the RGB LED act like a traffic light: red, green, yellow.

### Challenge 2: Automatic rainbow

Without the knob, make the colour go round the colour wheel by itself, smoothly. *Hint: use three* `for` *loops, one for each part of the wheel.*

### Challenge 3: Breathing light

Make the RGB LED "breathe": slowly fade in and out in one colour. Then make a button on pin 13 change which colour it breathes in.

### Bonus: Mood lamp

Use the photoresistor from Lesson 4: when the room gets dark, the RGB LED fades on in a warm orange colour.

## 7. Quiz

### Check what you learned

1. What does `analogWrite(LED, 0)` do? And `analogWrite(LED, 255)`?

2. In your own words, how does PWM make an LED look dim?

3. Which `setColor` values give pure blue?

**Tidy up:** unplug the USB cable. Keep the RGB LED and the potentiometer in place if you have room. Put the three traffic light LEDs back in the kit.